

---

# wev: Distilling LLM Browser Agents into Open, Local System-One Decision Models

---

AUTHOR NAME<sup>1</sup>

## Abstract

System-One decision models answer typed questions about a free-form state with a probability for every declared option, in one forward pass and without generating text, but the open models of this kind are trained on general decision data and reach at most 21% step success on browser steps from unseen websites. We observe that the typed interface between a browser agent and its System One is itself a source of supervision: a prompted LLM placed behind it while the agent works on live websites produces training data in exactly the decision model’s format, and an LLM judge reading each episode’s final page decides which episodes to keep. We combine this interface distillation with Mind2Web and NNetNav demonstrations converted to the same format, after auditing NNetNav’s stopping labels, and with general decision corpora. The resulting open models, WEV-1.7b, WEV-4b and WEV-8b, build on Qwen3 base models. WEV-4b answers 76% of browser steps on unseen websites correctly and stays competitive on general decision benchmarks. As the System One of an open browser agent, it completes as many held-out live-website tasks as its LLM teacher, at 15–320 ms per decision on one consumer GPU.

## 1. Introduction

Language-model agents act on the web by reading a page and choosing what to do next (Yao et al., 2023; Zhou et al., 2024; Deng et al., 2023; He et al., 2024). Most of their steps do not require writing: they require a choice among a small set of operations and, for the chosen operation, a choice among the elements on the page. Generative LLMs make these choices by producing text that must then be parsed, which costs a full autoregressive call per step and leaves the confidence of the choice implicit. System-One

decision models (TypeSafe AI, 2026) replace this with a typed interface: the caller declares questions and their options at request time, and the model returns a probability for every option in one forward pass, with nothing to parse and nothing that can be malformed. The open agent jev-ultrafast (Browser Use, 2026) is built on this interface: at each step it asks a hosted System One which operation to perform and which listed element to target.

Open decision models have followed quickly. Kev (kev contributors, 2026) adapts Qwen base models with LoRA and a pointer head, Laya (Laya contributors, 2026) fine-tunes ModernBERT (Warner et al., 2024), and this-that-model (Cheng et al., 2026) reads a 2B model’s answer from a designated position; work on selective control (Wu & Lim, 2026) and on read-out failure modes (Sun & Xu, 2026) studies how such models behave inside agents. All of these models learn from general decision data, meaning classification, question answering and rule records with short states. A browser step is different in kind. Its state is a page of text and a list of 8–40 candidate elements, about 3,200 tokens in our data, and its answer is grounded in that page rather than in world knowledge or in a rubric. Scored on identical requests, Kev-4B and Kev-8B get 21% and 19% of steps on unseen websites fully right and Laya under 1% (Section 5.1).

The obstacle to closing this gap is supervision. Web demonstrations exist (Deng et al., 2023; Murty et al., 2025; Lù et al., 2024), but they are recorded in other action formats, they rarely contain the stopping and blocking decisions an agent needs, and the stopping labels they do contain can be wrong. Collecting new human demonstrations in the decision format is slow. Our key observation is that the decision interface itself makes supervision cheap: if a prompted LLM answers the agent’s typed requests while the agent works on live websites, every logged request–answer pair is already a training example in the student’s input and output format, with no translation between action spaces. What remains is to decide which episodes to trust, and an LLM judge (Zheng et al., 2023) that reads the final page can make that decision. We call this procedure *interface distillation*.

We build WEV on this observation. A decision model following Kev’s design is trained on three sources: episodes from

---

<sup>1</sup>AFFILIATION. Correspondence to: AUTHOR NAME <EMAIL>.

interface distillation with qwen3-max (Yang et al., 2025) as the teacher; Mind2Web and NNetNav steps converted to the agent’s request format, with NNetNav’s stopping labels audited by a judge; and public general decision corpora that keep the model a general-purpose System One. Our contributions are:

- We show that open System-One models trained on general decision data do not transfer to browser steps, and release a conversion of Mind2Web and NNetNav into the request format of an open browser agent, with a single scorer for decision models (Section 5.1).
- We introduce interface distillation, which turns an LLM-backed agent run on live websites into judge-verified training data for a non-generative decision model, and an audit of NNetNav’s stopping labels that overturns 31% of its DONE labels (Section 3).
- We release WEV-1.7b, WEV-4b and WEV-8b. They reach 76% step success on unseen websites, stay competitive on general decision benchmarks, match their teacher’s task success on 153 held-out live-website tasks, and answer in 15–320 ms on one consumer GPU (Section 5).

Code, data builders, models and per-request evaluation outputs are public.

## 2. Related Work

**System-One decision models.** The hosted Jev model (TypeSafe AI, 2026) introduced a model class that returns typed, calibrated answers instead of text; its training method is unpublished. Open models share its interface but differ in backbone and data. Kev (kev contributors, 2026) attaches LoRA adapters (Hu et al., 2021) and a pointer head to Qwen base models and trains on ten public classification sources plus rule-composition records; Laya (Laya contributors, 2026) fine-tunes a bidirectional encoder (Devlin et al., 2019; Warner et al., 2024) with a small decision head; this-that-model (Cheng et al., 2026) reads answers from a designated position of a 2B decoder. REFLEX (Wu & Lim, 2026) places a hosted decision model inside an LLM agent and escalates to a strong LLM when confidence is low, and Sun & Xu (2026) show that decision heads can follow an option’s name rather than the rubric bound to it. None of these trains or evaluates a decision model on browser steps, and none derives training data from an agent’s own decision traffic.

**Web agents and web data.** Benchmarks for web agents range from simulated environments (Liu et al., 2018; Yao et al., 2022) to self-hosted websites (Zhou et al., 2024; Koh et al., 2024) and live websites (He et al., 2024). Mind2Web

(Deng et al., 2023) and WebLINX (Lù et al., 2024) record human demonstrations on real sites, and NNetNav (Murty et al., 2025) collects trajectories through unsupervised interaction with live sites. Most agents built on these resources generate actions with an LLM (Yao et al., 2023; Zheng et al., 2024a; Gur et al., 2024). We instead cast each step as typed questions and answer them with a non-generative model, converting these resources into that format.

**Distilling agents into smaller models.** Knowledge distillation (Hinton et al., 2015) and rationale distillation (Hsieh et al., 2023) transfer behaviour from large to small models, and agent fine-tuning on LLM-generated trajectories improves small generative agents (Chen et al., 2023a; Zeng et al., 2023; Schick et al., 2023). Imitation learning from a queryable expert (Ross et al., 2011) motivates collecting labels on the learner’s own state distribution. Interface distillation differs in its target and its supervision channel: the student generates nothing, and the teacher is observed through the same typed interface the student will serve, so the logged traffic needs no conversion.

**Calibration, option bias and routing.** Decision models are useful to the extent that their probabilities can be acted on. Modern networks are often miscalibrated (Guo et al., 2017); language models carry priors over answer strings (Zhao et al., 2021) and option positions (Zheng et al., 2024b), though they can also estimate their own correctness (Kadavath et al., 2022). Calibrated confidence is the basis of cascades and routers that send only hard inputs to expensive models (Chen et al., 2023b; Ong et al., 2025; Wu & Lim, 2026). We report calibration for every benchmark and study the stopping decision, where a threshold on one probability trades early stops against missed ones.

## 3. Method

### 3.1. Browser steps as typed decisions

A request consists of a state  $s$  and questions  $q_1, \dots, q_m$ , where question  $q_j$  declares options  $O_j = \{o_{j1}, \dots, o_{jK_j}\}$ , each with a name and a short rubric. A decision model returns a distribution  $p_\theta(\cdot \mid s, q_j)$  over  $O_j$  for every question. In a browser step,  $s$  contains the goal, the page URL and title, up to 6,000 characters of visible page text, the candidate elements with their roles and labels, and the last ten actions. The first question asks for the operation (CLICK, TYPE\_TEXT, SELECT, SCROLL\_UP, SCROLL\_DOWN, WAIT, DONE or BLOCKED); for each operation that needs a target, a second question lists the candidate elements as [index] label options. The agent takes the most probable operation and, if needed, its most probable target; typed text comes from a separate text model. A step is *successful* when both the operation and the target match the reference.

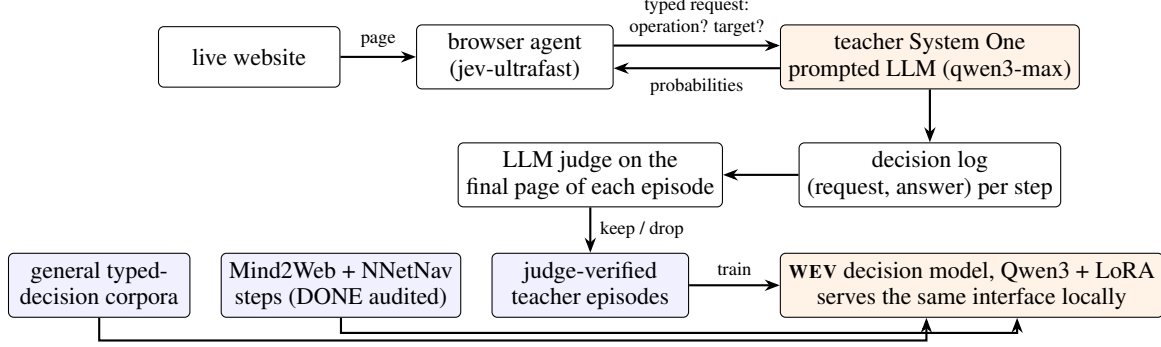


Figure 1. Interface distillation. A browser agent sends typed requests to a System One served by a prompted LLM while it works on live websites; every request and answer is logged in the decision model’s own format, and an LLM judge reading each episode’s final page decides which episodes are kept. The student WEV trains on these episodes together with converted demonstrations and general decision corpora, and then serves the same interface locally.

### 3.2. Decision model

We follow Kev’s design (kev contributors, 2026) and adapt its code. The state is written once, and each question follows it as its own branch, with options delimited by reserved tokens of the Qwen vocabulary:

```

<state> goal, page, elements, actions
<q> operation? <opt>CLICK</opt> ... <decide>
<q> CLICK target? <opt>[1] Search</opt> ... <decide>
    
```

Delimiter strings that appear in user text are rewritten, so text cannot forge structure.

**Branch mask.** Every token of a question branch attends causally to the state and to its own branch and never to another question, and each branch restarts its positions after the state. The model therefore answers all  $m$  questions in one forward pass over  $|s| + \sum_j |q_j|$  tokens, and neither question order nor packing changes an answer; our tests confirm this to fp32 noise.

**Backbone and read-out.** The backbone is a Qwen3 base model (Yang et al., 2025) without its vocabulary head, so it cannot generate text, adapted with LoRA of rank 16 on every attention and MLP projection. Let  $h_j^{\text{dec}}$  be the final hidden state of question  $j$ ’s `<decide>` token and  $h_{jk}$  that of option  $k$ ’s closing `</opt>` token. A pointer head projects both into  $d_p = 256$  dimensions and scores their scaled dot product,

$$z_{jk} = \frac{1}{\sqrt{d_p}} (W_k h_{jk} + b_k)^\top (W_q h_j^{\text{dec}} + b_q), \quad (1)$$

$$p_\theta(o_{jk} \mid s, q_j) = \text{softmax}_k z_{jk}. \quad (2)$$

Because the options are read from their own hidden states, the same model answers questions whose options it has never seen, which is what lets one network serve both general decisions and element lists that change on every page.

**Long states.** A page can exceed the context used in training, and truncating the encoded sequence would cut the element list, which carries the targets. We instead shrink the state before encoding, one step at a time, until it fits: the page text is cut by a quarter (never below 200 characters), then the URL is cut to 300 characters, then the per-element lists of select options are dropped, and finally the action history is reduced to the last three actions. The question branches, and with them every candidate target, are never shortened. The released models accept a state of up to 4,096 tokens and question branches of up to 8,192 tokens, so an open date picker with hundreds of options is still answered.

### 3.3. Interface distillation

**Teacher behind the interface.** We serve a prompted LLM, qwen3-max, as a System One. For each typed request the server renders the state and the questions as text, asks the LLM for one operation and one target key with a one-sentence reason, and returns a distribution that puts 0.97 on that choice, so the agent acts on it. The prompt adds safety rules that override the goal: never sign in, register, buy, book, post, send messages or submit personal information, and answer BLOCKED on CAPTCHAs, unusual-traffic pages and login walls (Section A). The server logs every request with the teacher’s choice as its label, keyed by the episode.

**Episodes on live websites.** We run jev-ultrafast with the teacher on 1,402 training tasks and hold out 153 evaluation tasks. Tasks pair NNetNav goals with their live sites, Wikipedia look-ups and Google Flights searches. Each episode runs in its own headless browser with a 60-action budget and a 300-second timeout.

**Keeping what the teacher got right.** An LLM judge (DeepSeek-V4.1-Flash) reads the goal and the state at the teacher’s final DONE decision and decides whether the goal is achieved. We keep every decision of an episode whose

Table 1. Episodes of the two teacher collections by outcome, and the decisions kept for training and development.

|                                        | First       | Second      |
|----------------------------------------|-------------|-------------|
| DONE, confirmed by the judge (kept)    | 173         | 103         |
| DONE, rejected by the judge            | 94          | 61          |
| BLOCKED on a blocking page (kept)      | 186         | 215         |
| BLOCKED in a loop (dropped)            | 341         | 501         |
| Budget exhausted (dropped)             | 121         | 165         |
| Harness error, prefix kept / too short | 39 / 101    | 25 / 89     |
| Other errors (dropped)                 | 144         | 318         |
| Decisions kept, train / dev            | 2,548 / 199 | 2,274 / 248 |

DONE the judge confirms. We also keep every decision up to a BLOCKED on a page that blocked the agent, since recognising a wall is itself a decision the student needs. For an episode cut short by the browser harness, we keep every decision but the last. Episodes that loop or exhaust their budget are dropped, as are repeated states within an episode, and tasks are split between training and development so that no task appears in both. Table 1 summarises the two collections: the first ran every training task, and the second reran the tasks the first had not solved.

The judge matters because the teacher’s own stopping decisions are unreliable. Across both collections the judge rejected 155 of the teacher’s 431 DONE claims (36%). Keeping those episodes would teach the student to stop early, which is the error it already makes most often (Section 5.4).

**Harness failures.** In the first collection, 35% of episodes ended in errors, most of them unrelated to the teacher. The harness left one browser daemon per episode running, and thousands of them exhausted the machine; separately, a page read during navigation aborted the episode. Both are fixed in the released collector, and every end-to-end number we report uses the fixed harness.

### 3.4. Converted demonstrations and a stopping-label audit

**Mind2Web.** Each recorded step becomes one request: the page’s visible text, a sample of 8–40 candidate elements containing the gold element, the task and the preceding actions, rendered exactly as jev-ultrafast renders its own requests. Splits are by website, so every test website is unseen in training (5,863 training, 586 development and 875 test requests; 873 after removing two whose element lists exceed the interface’s 255-option limit).

**NNetNav.** NNetNav (Murty et al., 2025) supplies what Mind2Web lacks: stopping (DONE), giving up (BLOCKED) and scrolling on live sites. Its trajectories come from unsupervised exploration, and its stopping labels are unreliable. We ask the judge, for every step, whether

Table 2. Audit of NNetNav’s stopping labels by an LLM judge. Relabelled: the goal was already achieved at a step labelled otherwise. Dropped: a DONE label at a step where the goal was not achieved.

|                    | Train  | Dev   | Test  |
|--------------------|--------|-------|-------|
| Kept as labelled   | 9,123  | 927   | 890   |
| Relabelled to DONE | 2,281  | 213   | 258   |
| DONE dropped       | 592    | 60    | 50    |
| Steps after audit  | 11,408 | 1,140 | 1,150 |

the goal is already achieved at that state. On the training split the judge rejects 592 of 1,898 DONE labels (31%) and finds the goal already achieved at 2,281 of the 10,102 other steps (23%). We relabel the latter as DONE and drop the former, since the correct next step there is unknown. The same audit applies to the development and test splits (Table 2), so that evaluation does not reward the original label noise; mislabelled test data can otherwise reorder models (Northcutt et al., 2021). Relabelling rather than dropping the missed stops matters for an agent: a step at which the goal is visibly achieved but the label says continue teaches the model to keep browsing after success.

### 3.5. General decisions and training

To keep WEV a general System One, the mixture (Table 3) adds Kev’s decision-v7 training split, 80% of the typed-decisions training split, and three public corpora: tasksource-jev, jev-distill-corpus-v3 and typed-decisions-synth. We use soft labels where a corpus provides them and remove every row whose state appears in the typed-decisions test split. Small, high-value sources are repeated within an epoch.

For a question with target distribution  $y_j$  over its options, which is one-hot for hard labels and the corpus’s distribution for soft labels, we minimise

$$\mathcal{L} = -\frac{1}{|B|} \sum_{(s,q_j) \in B} \sum_{k=1}^{K_j} y_{jk} \log p_{\theta}(o_{jk} | s, q_j), \quad (3)$$

where  $B$  is the set of questions in a micro-batch. Averaging over questions rather than requests gives a five-question typed-decisions case five times the weight of a one-question classification row, in proportion to the decisions it contains. We train for one epoch with AdamW, a peak learning rate of  $10^{-4}$  and a one-cycle schedule with 10% warm-up. Base weights stay frozen in bf16, and the adapters and head train in fp32.

Request lengths range from under 100 to about 6,000 tokens, so fixed-size batches would be dominated by padding. We shuffle the requests, sort them by length within windows of 4,096, and greedily pack each window into micro-batches whose padded size stays under a token budget of 6,000

Table 3. Training mixture after conversion. Weight is the number of repetitions per epoch. <sup>†</sup> WEV-4b only.

| Source                                           | License       | Rows   | Weight |
|--------------------------------------------------|---------------|--------|--------|
| Mind2Web (Deng et al., 2023)                     | CC BY 4.0     | 5,863  | 1      |
| NNetNav, DONE audited (Murty et al., 2025)       | Apache-2.0    | 11,408 | 1      |
| Teacher episodes, first collection               | model outputs | 2,548  | 3      |
| Teacher episodes, second collection <sup>†</sup> | model outputs | 2,274  | 3      |
| Kev decision-v7 (kev contributors, 2026)         | per source    | 12,576 | 2      |
| typed-decisions (80% of train)                   | Apache-2.0    | 960    | 8      |
| tasksources-jev                                  | mixed         | 57,024 | 1      |
| jev-distill-corpus-v3                            | Apache-2.0    | 50,000 | 1      |
| typed-decisions-synth                            | MIT           | 6,682  | 1      |

per GPU; the micro-batches are then shuffled again. The full plan for every epoch is fixed before training, which gives the learning-rate schedule its exact length. Under data parallelism each GPU takes every  $n$ -th micro-batch of the same plan, and the gradients of the adapters and the head, 34.3M parameters at 4B and 45.7M at 8B, are averaged across GPUs before each update. WEV-4b and WEV-8b train data-parallel on 8 NVIDIA H20 GPUs in 66 and 89 minutes, and WEV-1.7b on one GPU in 146 minutes. The released weights merge the adapter into the backbone.

## 4. Experimental Setup

**Benchmarks.** Kev decision-v7 (1,176 test requests) is Kev’s in-domain suite; Kev transfer-v4 (764) is out of domain for every model we evaluate; typed-decisions (400 cases with five questions each) covers four agent and operations workflows. Browser steps come from the Mind2Web (873) and NNetNav (1,150) test splits.

**Baselines.** Kev-4B, built on Qwen3.5-4B, and Kev-8B, built on Qwen3-8B, run through Kev’s own server in bf16. Laya and its typed-decisions specialist run through Laya’s Python package. All baselines run as published.

**Metrics.** Every model receives identical requests, and its most probable option is its answer. We report per-question accuracy on general benchmarks and step success on browser steps, the expected calibration error (ECE) (Guo et al., 2017) over all questions, DONE recall, and the premature-DONE rate, the share of not-finished steps answered DONE. WEV encodes states of up to 4,096 tokens; the 11 Mind2Web test requests beyond that count as wrong for WEV.

**End to end.** jev-ultrafast runs each of the 153 held-out tasks with the evaluated model as its System One, under the same budgets as collection. A task succeeds when the agent stops with DONE and the judge, reading the final page, agrees.

**Test protocol.** Test splits are held out from training and model selection with one exception: WEV-4b and WEV-8b

Table 4. Browser steps on the Mind2Web test split (873 requests, unseen websites). Step: operation and target both right; targets are chosen among 8–40 candidates. General decision models fail on page-grounded choices.

| Model                  | Step success | Operation acc. |
|------------------------|--------------|----------------|
| WEV-1.7b               | 68.2         | 88.1           |
| WEV-4b                 | <b>75.9</b>  | <b>91.2</b>    |
| WEV-8b                 | 75.5         | 90.3           |
| Kev-4B                 | 21.2         | 35.7           |
| Kev-8B                 | 19.0         | 73.3           |
| Laya (typed-decisions) | 0.7          | 13.1           |
| Laya                   | 0.0          | 2.5            |

Table 5. General typed decisions on test splits, per-question accuracy. WEV and Laya (typed-decisions) train on the typed-decisions training split; Kev and Laya do not. WEV is competitive but trails Kev on Kev’s own suites.

| Model                  | decision-v7 | transfer-v4 | typed-decisions |
|------------------------|-------------|-------------|-----------------|
| WEV-1.7b               | 81.1        | 65.5        | <b>79.5</b>     |
| WEV-4b                 | 80.6        | 73.8        | 79.4            |
| WEV-8b                 | 82.4        | 77.2        | 79.1            |
| Kev-4B                 | <b>88.2</b> | <b>82.1</b> | 65.1            |
| Kev-8B                 | 88.1        | 76.8        | 62.7            |
| Laya (typed-decisions) | 65.7        | 62.8        | 76.8            |
| Laya                   | 64.3        | 63.7        | 36.2            |

each had two final candidates, with and without the second teacher collection, and both candidates were read on test. The choice for WEV-4b rests on development and end-to-end results; the choice for WEV-8b was made after both test reads (Section 5.5).

## 5. Results

### 5.1. General decision models do not transfer to browser steps

Table 4 shows the transfer gap. Kev-8B chooses the right operation on 73% of steps, below the 79% of always answering CLICK, and the right element on far fewer. Laya’s context of 512 to 1,024 tokens holds only the start of a page. The failures are therefore of two kinds: a model without web training cannot ground an element choice in the page, and a model with a short context never sees most of the page. WEV gets three quarters of steps on unseen websites right; its remaining errors concentrate on targets (Table 7) and on rare operations (Section 5.7).

### 5.2. General decisions

Adding browser supervision does not cost WEV its general competence (Table 5). It matches or exceeds both Laya models on every general benchmark. Kev leads on its own

Table 6. Judge-verified task successes on 153 held-out live-website tasks with jev-ultrafast. The students match their teacher; differences are within run-to-run noise.

| System One                    | All<br>(153) | NNetNav<br>(123) | Wikipedia<br>(10) | Flights<br>(20) |
|-------------------------------|--------------|------------------|-------------------|-----------------|
| qwen3-max, prompted (teacher) | 27           | 13               | 10                | 4               |
| WEV-4b                        | 30           | 18               | 10                | 2               |
| WEV-8b                        | 28           | 18               | 10                | 0               |

Table 7. Expected calibration error over all questions (lower is better) and target accuracy on browser steps.

| Model    | ECE, all questions |              |         | Click-target acc. |             |
|----------|--------------------|--------------|---------|-------------------|-------------|
|          | transfer-v4        | Mind2Web     | NNetNav | Mind2Web          | NNetNav     |
| WEV-1.7b | 0.099              | 0.031        | 0.054   | 74.1              | 58.4        |
| WEV-4b   | <b>0.033</b>       | <b>0.024</b> | 0.059   | <b>81.4</b>       | <b>60.7</b> |
| WEV-8b   | 0.046              | 0.029        | 0.059   | <b>81.4</b>       | 60.2        |

suites: by six to eight points on decision-v7, whose training split all Qwen-based models here share, and by eight points on transfer-v4 over WEV-4b. At equal backbone the picture changes. WEV-8b and Kev-8B both build on Qwen3-8B and are level on transfer-v4 (77.2 against 76.8), while Kev-4B builds on the newer Qwen3.5-4B, which suggests that much of the 4B gap comes from the backbone. On typed-decisions, models that train on the dataset lead; an earlier 4B WEV trained without it scored 53.1% on its development split. Our score for Laya (typed-decisions), 76.8%, reproduces its published 76.6%.

### 5.3. End to end on live websites

Step accuracy measures single decisions against references; an agent must chain decisions on pages that change. With WEV as its System One, jev-ultrafast completes as many held-out tasks as with its teacher (Table 6). The differences between rows are within noise: two runs of WEV-8b variants with equal totals disagreed on 8 tasks in each direction. All three System Ones solve every Wikipedia task and few Google Flights tasks, which frequently answer automated browsing with a CAPTCHA. The student does this without generating text and on local hardware.

### 5.4. Calibration and the stopping decision

An agent acts on the most probable option, but a calibrated probability lets it act more selectively. The expected calibration error stays at or below 0.06 on browser steps and at or below 0.10 on out-of-domain decisions (Table 7). The most consequential browser decision is when to stop: stopping early ends a task unfinished, and stopping late wastes the budget. Figure 2 shows the trade-off obtained by accepting DONE only above a threshold on its probability. At a threshold of 0.5, WEV-4b recalls 82% of DONE steps and stops early on 8.5% of the others; at 0.8 the premature rate falls to 3.8% while recall falls to 67%. WEV-8b reaches

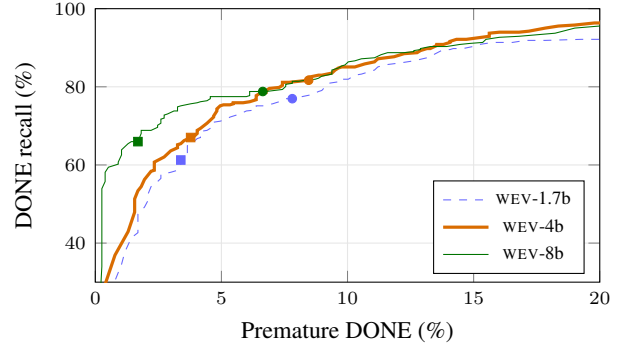


Figure 2. Stopping trade-off on the NNetNav test split (382 DONE steps, 768 others). Each curve accepts DONE only when its probability exceeds a threshold swept from 0.05 to 0.99; circles mark 0.5 and squares 0.8.

Table 8. Paired live-website outcomes when the second teacher collection is added. Gained: solved only with it; lost: solved only without it.

|        | Both | Gained | Lost | Total with |
|--------|------|--------|------|------------|
| WEV-4b | 20   | 10     | 1    | 30         |
| WEV-8b | 20   | 8      | 8    | 28         |

1.7% premature stops at 66% recall, or 2.9% at 73% with a threshold of 0.7. The curves are smooth, so a deployment can choose its operating point from development data rather than accept the most probable operation, which answers DONE whenever it outranks the other seven operations and so behaves like a threshold below 0.5.

### 5.5. What helped and what did not

**The second teacher collection.** Adding 2,274 decisions from tasks the first collection had not solved raised WEV-4b’s live-website successes from 21 to 30. Paired by task (Table 8), 10 tasks were gained and 1 was lost, an asymmetry unlikely under run-to-run noise (exact McNemar test,  $p = 0.012$ ).

Its test accuracy stayed within 0.6 points on every benchmark. For WEV-8b the same data left live-website successes at 28, with 8 tasks gained and 8 lost, and lowered transfer-v4 by 1.8 points, so WEV-8b keeps the first mixture.

**A second epoch.** Training WEV-4b for two epochs changed development accuracy by less than one point on every benchmark (transfer-v4 74.6 to 75.1, decision-v7 81.9 to 82.6, typed-decisions 79.7 to 79.2, Mind2Web step success 80.0 to 79.3) at twice the cost.

**Structural variants.** On 1.7B, with an earlier mixture, three seeds and development splits, we tested a set head, which runs a two-layer permutation-equivariant transformer

Table 9. Median in-process latency on one RTX 5090 (bf16, one request at a time). All questions of a request share one forward pass.

| Model    | decision-v7<br>1 q., 180 tok. | typed-decisions<br>5 q., 380 tok. | Browser step<br>2 q., 3,200 tok. |
|----------|-------------------------------|-----------------------------------|----------------------------------|
| WEV-1.7b | 10 ms                         | 13 ms                             | 91 ms                            |
| WEV-4b   | 15 ms                         | 25 ms                             | 217 ms                           |
| WEV-8b   | 21 ms                         | 37 ms                             | 322 ms                           |

Table 10. Recall of each reference operation on the browser test splits (%). Rare operations are rarely predicted.

| Split    | Operation         | Steps | WEV-1.7b | WEV-4b | WEV-8b |
|----------|-------------------|-------|----------|--------|--------|
| Mind2Web | CLICK             | 687   | 96       | 96     | 95     |
|          | SELECT            | 65    | 65       | 88     | 88     |
|          | TYPE.TEXT         | 110   | 63       | 75     | 71     |
| NNetNav  | CLICK             | 478   | 82       | 80     | 84     |
|          | DONE              | 382   | 80       | 85     | 80     |
|          | TYPE.TEXT         | 201   | 79       | 76     | 76     |
|          | SCROLL (up, down) | 60    | 0        | 0      | 3      |
|          | BLOCKED           | 29    | 3        | 3      | 3      |

over the option states before scoring, and truncation of the backbone to its first 18 of 28 layers. Mind2Web step success was  $69.0 \pm 2.7$  for the pointer head at full depth,  $66.5$  for the set head at full depth and  $68.5 \pm 0.2$  for the set head on 18 layers, which ran 23% faster. A later control run on identical hardware put the cost of truncation at about three points, and the set head did not help at full depth, so the released models use the pointer head and every layer.

### 5.6. Efficiency

Because all questions share one forward pass, five questions cost less than twice one (Table 9), and a browser step’s cost is dominated by encoding the page. On a 16 GB Apple M5 laptop with the PyTorch MPS backend in fp32, WEV-1.7b loads in about 30 s and answers a one-question decision in 77 ms.

### 5.7. Failure analysis

Operations that are rare in training are rarely predicted (Table 10). On NNetNav test steps, recall is 76–85% for CLICK, TYPE.TEXT and DONE but 3% for BLOCKED (29 steps) and at most 4% for scrolling (60 steps), for all three sizes. On Mind2Web, TYPE.TEXT recall ranges from 63% (WEV-1.7b) to 75% (WEV-4b), against 95–96% for CLICK. Target choice remains harder on NNetNav (58–61%) than on Mind2Web (74–81%), since NNetNav pages come from live exploration and have longer, noisier element lists. On live tasks, most failures are pages that block automated browsing and tasks that exceed the action budget.

## 6. Discussion

**Why the interface is the right place to distil.** A generative agent’s supervision lives in its own action format: tool calls, free text, or coordinates. Converting that format into typed questions requires choosing, for every step, which options the student should have seen and how to name them, and every such choice can introduce a mismatch between training and deployment. At the decision interface the agent has already made these choices, so the teacher and the student see byte-identical requests. The teacher’s reasons are discarded, and only the choice is kept, which is all that a decision model can learn from a hard-labelled step.

**What bounds the student.** Interface distillation cannot exceed its teacher on the states the teacher visits, and the judge only removes the teacher’s failures; it does not correct them. The student matches rather than surpasses its teacher end to end, and its gains from the second collection came from tasks that the first had failed, which suggests that coverage of hard states, not the number of easy decisions, is what limits the student. The judge is a second point of failure: it reads a page, not the world, and it can accept a stop on a page that only resembles the answer.

## 7. Limitations

Our live evaluation has 153 tasks on websites that change between runs, which gives wide intervals; we therefore compare versions with paired tests. Browser targets are chosen among the elements the agent lists, not all elements on the page, so our Mind2Web numbers are not comparable to its leaderboard. The teacher’s labels are hard choices, so the student learns the teacher’s decisions rather than its uncertainty, and interface distillation inherits the teacher’s blind spots; the judge that filters episodes is itself an LLM. Two test reads informed the choice of WEV-8b. We cover English only and could not compare with the hosted Jev model. Parts of the training data carry their own terms: several tasksource-jev source tasks are licensed for research only, and the teacher episodes are outputs of qwen3-max.

## 8. Conclusion

The typed interface between an agent and its System One is a supervision channel: logging a prompted LLM behind it on live websites, and keeping what a judge confirms, yields training data in the decision model’s own format. Combined with converted demonstrations whose stopping labels we audited, this closes the gap that general decision models leave on browser steps. WEV answers three quarters of steps on unseen websites, stays competitive on general decisions, and completes as many live tasks as its teacher, locally and in tens to hundreds of milliseconds per decision. Collecting

teacher labels on the states the student itself reaches, in the manner of DAgger (Ross et al., 2011), is the natural next step.

## Impact Statement

This work makes browser agents cheaper to run and removes their dependence on a hosted decision API, which lowers the barrier to automating web tasks. Automated browsing can be misused, for example to scrape sites against their terms or to act on accounts without consent. Our teacher prompt forbids signing in, purchasing, posting, messaging and submitting personal information, and the student learns to answer BLOCKED on CAPTCHAs and login walls, but these behaviours are learned, not guaranteed. We release the models as research artifacts under Apache-2.0, note the research-only terms of part of the training data, and recommend human oversight for any deployment that acts on behalf of users.

## References

- Browser Use. jev-ultrafast. <https://github.com/browser-use/jev-ultrafast>, 2026.
- Chen, B., Shu, C., Shareghi, E., Collier, N., Narasimhan, K., and Yao, S. Fireact: Toward language agent fine-tuning, 2023a. URL <https://arxiv.org/abs/2310.05915>.
- Chen, L., Zaharia, M., and Zou, J. Frugalgpt: How to use large language models while reducing cost and improving performance, 2023b. URL <https://arxiv.org/abs/2305.05176>.
- Cheng, Z., Dai, W., and Sun, J. this-that-model-1.0: A typed decision model that decides in 30 ms, for a millionth of a cent, 2026. URL <https://arxiv.org/abs/2609.23886>.
- Deng, X., Gu, Y., Zheng, B., Chen, S., Stevens, S., Wang, B., Sun, H., and Su, Y. Mind2web: Towards a generalist agent for the web. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2306.06070>. arXiv:2306.06070.
- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019. URL <https://arxiv.org/abs/1810.04805>.
- Guo, C., Pleiss, G., Sun, Y., and Weinberger, K. Q. On calibration of modern neural networks. In *International Conference on Machine Learning*, 2017. URL <https://arxiv.org/abs/1706.04599>. arXiv:1706.04599.
- Gur, I., Furuta, H., Huang, A., Safdari, M., Matsuo, Y., Eck, D., and Faust, A. A real-world webagent with planning, long context understanding, and program synthesis. In *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2307.12856>. arXiv:2307.12856.
- He, H., Yao, W., Ma, K., Yu, W., Dai, Y., Zhang, H., Lan, Z., and Yu, D. Webvoyager: Building an end-to-end web agent with large multimodal models. In *Annual Meeting of the Association for Computational Linguistics*, 2024. URL <https://arxiv.org/abs/2401.13919>. arXiv:2401.13919.
- Hinton, G., Vinyals, O., and Dean, J. Distilling the knowledge in a neural network, 2015. URL <https://arxiv.org/abs/1503.02531>.
- Hsieh, C.-Y., Li, C.-L., Yeh, C.-K., Nakhost, H., Fujii, Y., Ratner, A., Krishna, R., Lee, C.-Y., and Pfister, T. Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes. In *Findings of the Association for Computational Linguistics: ACL 2023*, 2023. URL <https://arxiv.org/abs/2305.02301>. arXiv:2305.02301.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. Lora: Low-rank adaptation of large language models, 2021. URL <https://arxiv.org/abs/2106.09685>.
- Kadavath, S., Conerly, T., Askell, A., Henighan, T., Drain, D., Perez, E., Schiefer, N., Hatfield-Dodds, Z., DasSarma, N., Tran-Johnson, E., Johnston, S., El-Showk, S., Jones, A., Elhage, N., Hume, T., Chen, A., Bai, Y., Bowman, S., Fort, S., Ganguli, D., Hernandez, D., Jacobson, J., Kernion, J., Kravec, S., Lovitt, L., Ndousse, K., Olsson, C., Ringer, S., Amodei, D., Brown, T., Clark, J., Joseph, N., Mann, B., McCandlish, S., Olah, C., and Kaplan, J. Language models (mostly) know what they know, 2022. URL <https://arxiv.org/abs/2207.05221>.
- kev contributors. kev: decision models on qwen base models. <https://github.com/jaredpalmer/kev>, 2026.
- Koh, J. Y., Lo, R., Jang, L., Duvvur, V., Lim, M. C., Huang, P.-Y., Neubig, G., Zhou, S., Salakhutdinov, R., and Fried, D. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks. In *Annual Meeting of the Association for Computational Linguistics*, 2024. URL <https://arxiv.org/abs/2401.13649>. arXiv:2401.13649.
- Laya contributors. Laya: a non-autoregressive system 1 decision engine. <https://github.com/NandhaKishorM/laya>, 2026.

- Liu, E. Z., Guu, K., Pasupat, P., Shi, T., and Liang, P. Reinforcement learning on web interfaces using workflow-guided exploration. In *International Conference on Learning Representations*, 2018. URL <https://arxiv.org/abs/1802.08802>. arXiv:1802.08802.
- Lù, X. H., Kasner, Z., and Reddy, S. Weblinx: Real-world website navigation with multi-turn dialogue, 2024. URL <https://arxiv.org/abs/2402.05930>.
- Murty, S., Zhu, H., Bahdanau, D., and Manning, C. D. Nnetnav: Unsupervised learning of browser agents through environment interaction in the wild, 2025. URL <https://arxiv.org/abs/2410.02907>.
- Northcutt, C. G., Athalye, A., and Mueller, J. Pervasive label errors in test sets destabilize machine learning benchmarks. In *Advances in Neural Information Processing Systems*, 2021. URL <https://arxiv.org/abs/2103.14749>. arXiv:2103.14749.
- Ong, I., Almahairi, A., Wu, V., Chiang, W.-L., Wu, T., Gonzalez, J. E., Kadous, M. W., and Stoica, I. Routellm: Learning to route llms with preference data, 2025. URL <https://arxiv.org/abs/2406.18665>.
- Ross, S., Gordon, G. J., and Bagnell, J. A. A reduction of imitation learning and structured prediction to no-regret online learning. In *International Conference on Artificial Intelligence and Statistics*, 2011. URL <https://arxiv.org/abs/1011.0686>. arXiv:1011.0686.
- Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T. Toolformer: Language models can teach themselves to use tools, 2023. URL <https://arxiv.org/abs/2302.04761>.
- Sun, Y. and Xu, J. Type-safe is not error-free: A constrained decision head follows the option name, not the rubric bound to it, 2026. URL <https://arxiv.org/abs/2609.26758>.
- TypeSafe AI. Jev: a system one model for typed, calibrated decisions. <https://typesafe.ai>, 2026.
- Warner, B., Chaffin, A., Clavié, B., Weller, O., Hallström, O., Taghadouini, S., Gallagher, A., Biswas, R., Ladhak, F., Aarsen, T., Cooper, N., Adams, G., Howard, J., and Poli, I. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference, 2024. URL <https://arxiv.org/abs/2412.13663>.
- Wu, T. and Lim, W. Y. B. Reflex with jev for efficient selective control in llm agents, 2026. URL <https://arxiv.org/abs/2609.26532>.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., and Qiu, Z. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Yao, S., Chen, H., Yang, J., and Narasimhan, K. Webshop: Towards scalable real-world web interaction with grounded language agents, 2022. URL <https://arxiv.org/abs/2207.01206>.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafraan, I., Narasimhan, K., and Cao, Y. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL <https://arxiv.org/abs/2210.03629>. arXiv:2210.03629.
- Zeng, A., Liu, M., Lu, R., Wang, B., Liu, X., Dong, Y., and Tang, J. Agenttuning: Enabling generalized agent abilities for llms, 2023. URL <https://arxiv.org/abs/2310.12823>.
- Zhao, T. Z., Wallace, E., Feng, S., Klein, D., and Singh, S. Calibrate before use: Improving few-shot performance of language models. In *International Conference on Machine Learning*, 2021. URL <https://arxiv.org/abs/2102.09690>. arXiv:2102.09690.
- Zheng, B., Gou, B., Kil, J., Sun, H., and Su, Y. Gpt-4v(ision) is a generalist web agent, if grounded, 2024a. URL <https://arxiv.org/abs/2401.01614>.
- Zheng, C., Zhou, H., Meng, F., Zhou, J., and Huang, M. Large language models are not robust multiple choice selectors. In *International Conference on Learning Representations*, 2024b. URL <https://arxiv.org/abs/2309.03882>. arXiv:2309.03882.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., and Stoica, I. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2306.05685>. arXiv:2306.05685.
- Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Ou, T., Bisk, Y., Fried, D., Alon, U., and Neubig, G. Webarena: A realistic web environment for building autonomous agents, 2024. URL <https://arxiv.org/abs/2307.13854>.

## A. Teacher and Judge Prompts

**Teacher.** The teacher server renders each request as the goal, URL, title, up to 6,000 characters of visible page text, the element list with roles, labels, states and allowed operations, the recent actions, the operation options with their rubrics, and the valid target keys per operation, followed by the agent’s own rules and these safety rules:

Never sign in, register, buy, book, pay, post, comment, send messages, or submit personal information. If the next step would need any of those, a CAPTCHA, an “unusual traffic” page, or a login wall: choose BLOCKED. If the goal is already visibly satisfied: choose DONE. Do not keep browsing after that.

The teacher replies with a JSON object holding one operation key, one valid target key or null, and a one-sentence reason.

**Judge.** The judge receives the goal and the current browser state and is instructed:

Decide whether the goal has already been fully achieved, so that the correct next step is to stop and report completion. “done” is true only if the current page and the actions taken already satisfy every part of the goal. For information-seeking goals, done is true when the requested information is visible on the current page or was clearly reached by the actions taken. For goals that require an action, done is true only if that action has visibly been completed. If a required part is missing, or the page does not show the information yet, done is false.

It replies with a JSON object holding a boolean and a one-sentence reason.

## B. Hyperparameters

Table 11. Training hyperparameters of the released models.

|                                             | WEV-1.7b                               | WEV-4b           | WEV-8b        |
|---------------------------------------------|----------------------------------------|------------------|---------------|
| Base model                                  | Qwen3-1.7B-Base                        | Qwen3-4B-Base    | Qwen3-8B-Base |
| Layers                                      | 28                                     | 36               | 36            |
| LoRA rank / targets                         | 16 / all attention and MLP projections |                  |               |
| Pointer-head dimension $d_p$                | 256                                    |                  |               |
| Peak learning rate, schedule                | $10^{-4}$ , one-cycle, 10% warm-up     |                  |               |
| Epochs                                      | 1                                      |                  |               |
| Padded tokens per micro-batch and GPU       | 20,000                                 | 6,000            | 6,000         |
| GPUs (NVIDIA H20)                           | 1                                      | 8                | 8             |
| Training time                               | 146 min                                | 66 min           | 89 min        |
| Teacher collections                         | first                                  | first and second | first         |
| Maximum state / question tokens (inference) |                                        | 4,096 / 8,192    |               |

## C. Development Results

Table 12. Development results of the WEV-4b candidates (per-question accuracy; browser columns are step success; Teacher is the development split of the first collection).

| Candidate                            | decision-v7 | transfer-v4 | typed-dec. | Mind2Web | NNetNav | Teacher |
|--------------------------------------|-------------|-------------|------------|----------|---------|---------|
| First collection, 1 epoch            | 81.9        | 74.6        | 79.7       | 80.0     | 64.8    | 71.1    |
| First collection, 2 epochs           | 82.6        | 75.1        | 79.2       | 79.3     | 67.6    | 69.0    |
| Both collections, 1 epoch (released) | 81.5        | 72.6        | 79.7       | 77.2     | 66.7    | 74.1    |

## D. Release

Code, data builders and per-request evaluation outputs: <https://github.com/alanhuangyoo/wev>. Models: <https://huggingface.co/alanhuangya>. Package: `pip install wev-ai`.